Clock Domain Crossing

Clock domain crossing (CDC) refers to the transfer of signals from one clock domain to another in a digital system. But what is a clock domain? when can we say that these two parts of the design are in two different clock domains?

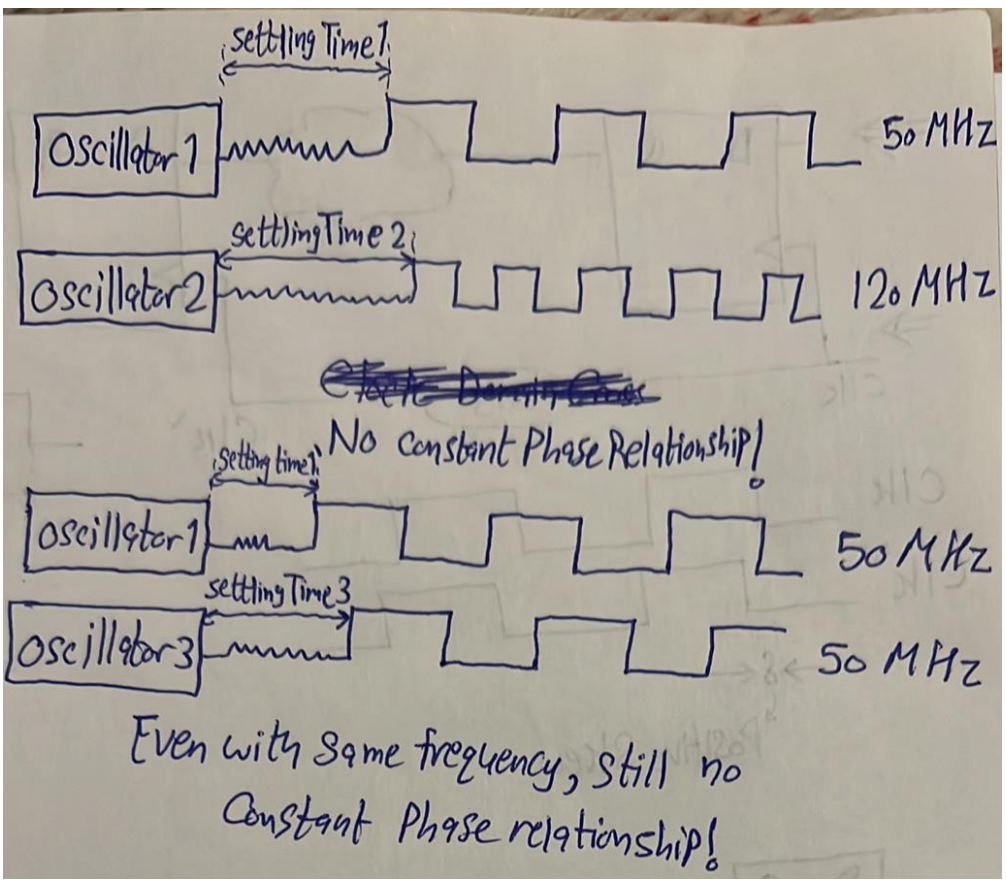
Two parts of the design are said to be in two different clock domains if they are operating with clocks that do not have a constant phase relationship. This can happen if the two clocks are generated from:

1. two different clock sources.
2. the same clock source but their frequencies do not have an integer relationship. That is:

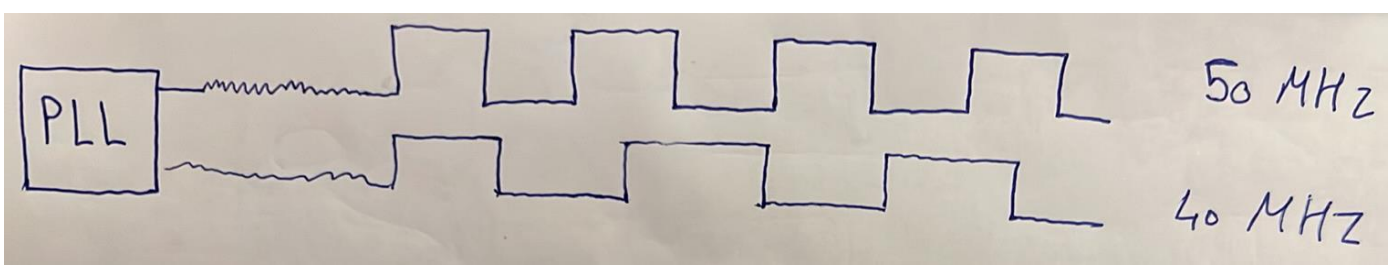
$$f_2 \neq nf_1 \text{ and } f_1 \neq nf_2 \ (n: integer)$$

But why if two clocks are generated from two different clock sources or the same clock source but with no integer relationship do not have a constant phase relationship?

1. Two different clock sources: because each clock source has something called "settling time" when it is powered on. This settling time is a random delay that a clock source takes when it is powered on until its output settles on the desired frequency. As a result, two different clock sources will have different settling times and accordingly it will be impossible to maintain a constant phase relationship between them. Even if the two clock sources are generating the same frequency, they will still do not have a constant phase relationship as each of them have a different settling time and hence sending data from one of them to the other is considered clock domain crossing! examples are shown below:



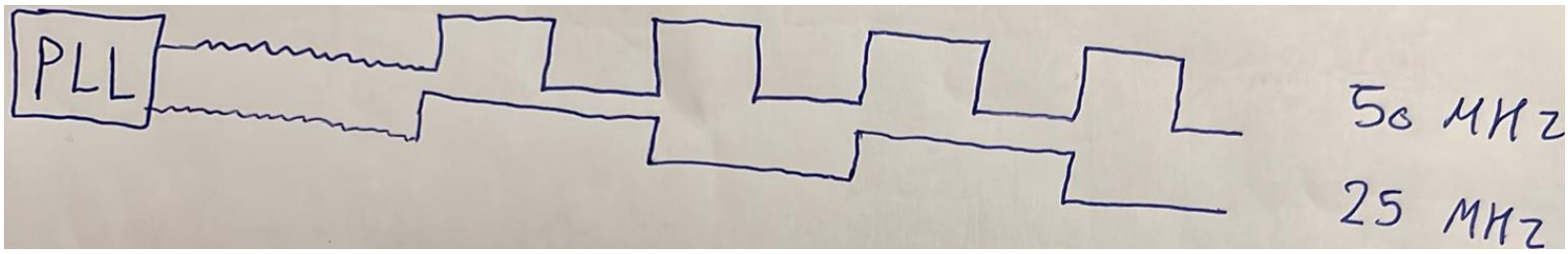
2. Same clock source but their frequencies do not have an integer relationship: this is trivial to understand, just check the example below. It is clear that the two clocks do not have a constant phase relationship although they have the same settling time since they are generated from the same clock source.



So, the only way for two clocks to have a constant phase relationship is to:

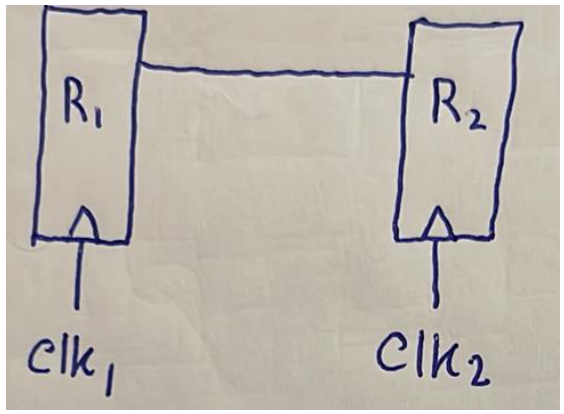
1. be generated from the same clock source.
2. have an integer relationship.

An example is shown below:

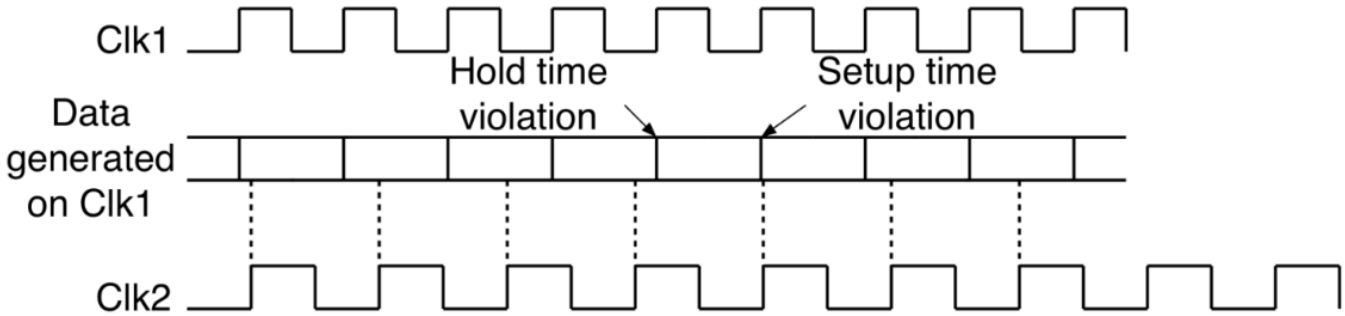


But why is CDC a thing we should take care of?

Consider the below setup. Register R_1 is sending data to register R_2. R_1 is operating with clock clk_1 and R_2 is operating with clock clk_2, and the two clocks are assumed to be in different clock domains, so this is CDC.



Since the two clocks do not have a constant phase relationship, then for sure setup and hold time violations will occur in R_2 in a cyclic manner (i.e. every now and then) as shown below!



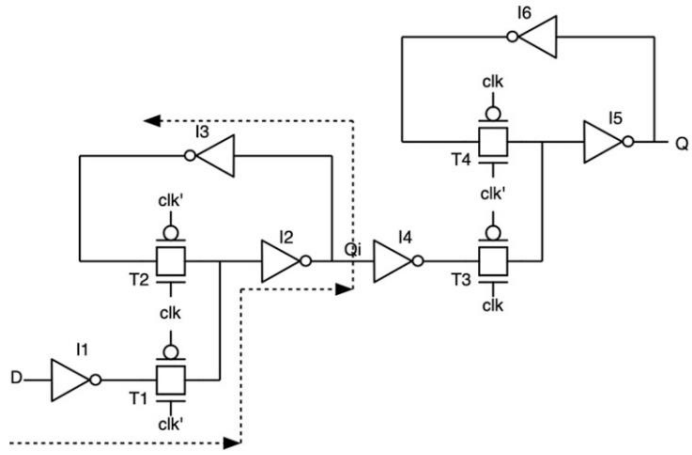
Thus, CDC causes timing violations which lead to metastability!

The questions now are:

1. How does a register behave when it goes metastable?
2. How often does metastability?
3. How long does a register take to get out of metastability?
4. How to handle/solve metastability?
5. How often do these violations occur after handling CDC?

How does a register behave when it goes metastable?

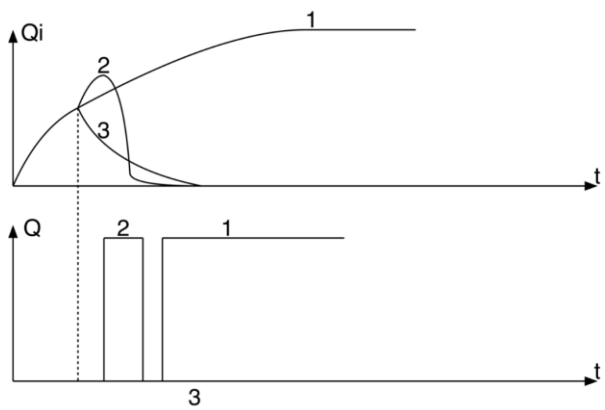
As we said in 3rd year, a flip flop internally is as shown below:



We also said that the setup time is:

$$t_{su} = t_{I1} + t_{T1} + t_{I2} + t_{I3}$$

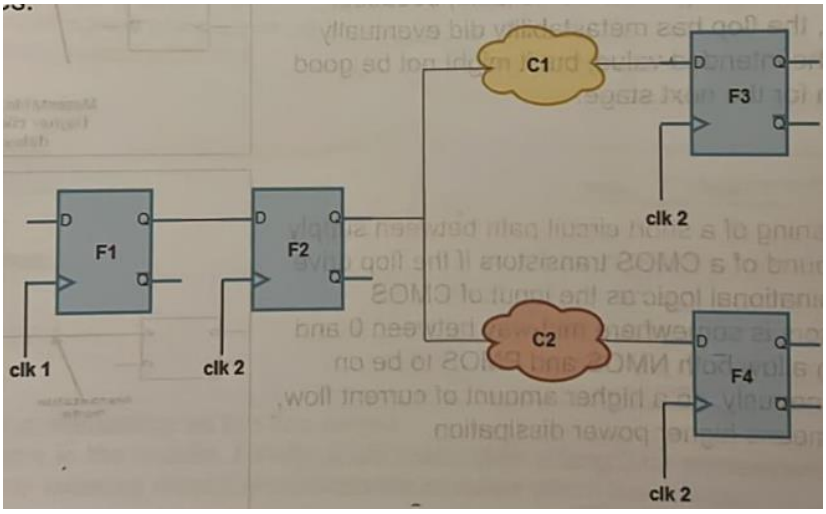
W olna en el setup time howa el wa2t el abl el clock edge el lazem a7afez 3ala el data tkoon sabta fih w ela momken mayet3emelahash latching be tare2a mazbota. Tab hena ana eh el momken yeb2a me5awefny en maye7salahash latching be tare2a mazbota lama el clock tegy? La2en lama el clock tegy el transmission gate T_1 hatefta7, fa momken el change el ana lesa 3amlo da mayel7a2sh yed5ol gowa. 5alas yeb2a ana lazem aghayar el input abl el clock edge be w2t kefaya such that eno ye3ady men el transmission gate T_1. Tab ma keda da ma3nah en t_{su} should be $t_{I1} + t_{T1}$ bs, leeh 5adna $t_{I2} + t_{I3}$ kaman m3ana? Fa fi 3rd year ma2olnash el sabab, delwa2ty b2a nefham el sabab. You need to watch video 1, but first watch video 2 to remember how two inverters in positive feedback behave.



So, we are now convinced that metastability is a failure in the output of a flip flop which might cause a complete failure in the whole pipeline.

Other than high t_{cq}, another problem that can happen due to metastability is propagation of different values at the same time.

Consider the example shown below.



Assume:

1) F2 enters metastability.

2) C1 combinational logic has a delay of 7 ns.

3) C2 combinational logic has a delay of 3 ns.

4) T_clk2 = 10 ns.

5) T_c2q of F2 = 1 ns, and if it enters metastability t_c2q becomes 3 ns.

6) F2's output is initially 0 and it is intended to change to 1.

7) t_setup of F3 & F4 = 1 ns.

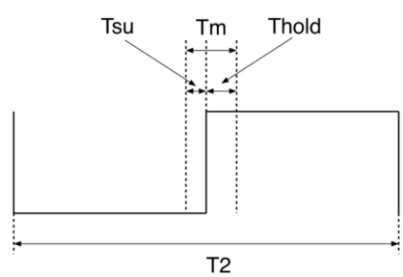
Lw makansh fih metastability, fa el mafrood el kan hy7sl en el output bta3 F2 da kan fel awel be 0, then haya5od 1 ns 3ashan yetla3 le 1. Then C1 haya5od 7 ns 3ashan ye evaluate el output beta3o based on el 1 el galo men F2 da, w yetala3 output 3la F3 b3d 8 ns, w fih t_setup be 1 ns, yeb2a keda 9 ns, wel clock edge hayegy 3and 10 ns, fa keda tamam F3 3amal capture lel value el sa7. Nafs el kalam el ne7ya el tanya, C2 haya5od 3 ns 3ashan ye evaluate el output beta3o based on el 1 el galo men F2 da, w yetala3 output 3la F4 b3d 4 ns, w fih t_setup be 1 ns, yeb2a keda 5 ns, wel clock edge lesa badry aslan hayegy 3and 10 ns, yeb2a F4 3amal capture lel value sa7.

However, lama ye7sal metastability b2a, el t_c2q bta3 F2 hayeb2a 3 ns. In other words, el output bta3 F2 hayo3od be 0 kteer, hywsl le 1 b3d 3 ns instead of 1 ns. Fa b3d 3 ns, C2 hay evaluate el output beta3o based on el 1 el galo da w ytl3 el output beta3o b3d 3 ns, y3ny 3and 6 ns, w fih 1 ns setup y3ny 7 ns, wel clock edge lesa 3and 10 ns, yeb2a F4 da hay capture el data sa7 3ady. On the other hand b2a, C1 hay evaluate el output beta3o based on el 1 el galo da b3d 7 ns, y3ny el output beta3o hayeb2a gahez b3d 10 ns, w lesa fih aslan 1 ns setup time. Keda eh el 7asal? F3 hy3ml capturing lel output bta3 C1 el kan based 3la 0 output bta3 F2, el howa el output el adeem, fi 7een F4 3ady hay capture el value el gededa el sa7 el heya based on 1 gaylo men F2. Thus, 7asal hena propagation of different values to F3 and F4. Yeb2a bsbb en el output bta3 F2 da da5al fi metastability, fi nodes shafeto 0 w fi nodes shafeto 1. Moreover, mesh bs hy7sl propagation le different values at the same time, da kaman momken el flip flops el b3d keda ye7sal fiha setup/hold time violations.

The question now is: how often does this metastability occurs, maho lw bete7sal masalan kol sana aw sanateen masalan mesh moshkela, mafeesh gehaz mabye7salsh fih failure y3ny, fa lw da router y3ny w kol sanateen ye7sal fih failure mesh moshkela yasedy neb2a ne3melo restart sa3et-ha. Fa 3ayzeen nshoof el metastability bete7sal kol 2ad eh.

How often does metastability?

For the clock shown below (of the capturing flip flop), the probability that metastability occurs is the probability that the data arrives within the setup and hold window.



Thus, the probability of metastability is:

$$P(metastability) = \frac{T_m}{T_2}$$

Notice that there is an implicit assumption here that the data is equally probable to arrive at any point during T_2, or in other words the random variable that describes the arrival time of the data has a uniform distribution. Does this assumption make sense? definitely, because the two clocks do not have a constant phase relationship and hence clock domain 1 can send clock domain 2 data at any time during T_2.

If the launching flip flop (i.e. clock domain 1) is sending data to clock domain 2 with rate f_1, then the rate of metastability is:

$$R(metastability) = \frac{T_m}{T_2} f_1 = \frac{T_m}{T_1 T_2}$$

Let's take typical values for T_m, T_1, T_2 to see whether this rate is high or low so that we can decide whether metastability occurs frequently or rarely:

$$T_m = 10\ ps$$

$$f_1 = 500\ MHz \rightarrow T_1 = 2\ ns$$

$$f_2 = 400\ MHz \rightarrow T_2 = 2.5\ ns$$

$$\therefore R(metastability) = 2\ million\ times\ per\ second$$

This is for sure unbearably high!

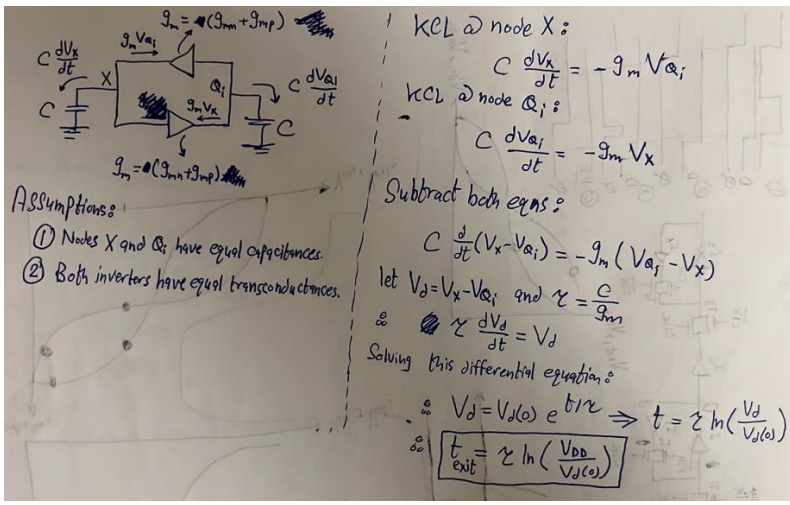
$$MTBF = \frac{1}{R} = 0.5\ us$$

MTBF (Mean Time Between Failures): time between two failures OR the time until the first failure occurs.

So, imagine you bought a new router and it failed just $0.5\ us$ after you powered it on and then keeps on failing every $0.5\ us$!

How long does a register take to get out of metastability?

- Watch video 3. Watch the first 12:15 minutes of this for the derivations. The derivation:



- Having a flip flop that entered metastability, the probability that this flip flop remains in metastability after time t from the clock edge is:

$$P(still\ metastable\ after\ time\ t\ from\ edge \mid metastability\ occured) = e^{-t/\tau}$$

Thus, the probability that a flip flop remains in metastability in general after time t from the clock edge is:

$$\begin{aligned} &P(metastability\ \cap still\ metastable\ after\ time\ t\ from\ edge) \\ &= P(metastability) * P(still\ metastable\ after\ time\ t\ from\ edge \mid metastability\ occured) \\ &= \frac{T_m}{T_2} * e^{-t/\tau} \end{aligned}$$

Let's take $t = T_2$, which means we waited for a complete clock cycle from the edge metastability occurred at.

$$\therefore P(metastability\ \cap still\ metastable\ after\ time\ T_2\ from\ edge) = \frac{T_m}{T_2} * e^{-T_2/\tau}$$

$$\therefore R(metastability\ \cap still\ metastable\ after\ time\ T_2\ from\ edge) = \frac{T_m}{T_1 T_2} * e^{-T_2/\tau}$$

If we took the previous values plus $\tau = 20\ ps$, we get:

$$P(metastability\ \cap still\ metastable\ after\ time\ T_2) = 2 * 10^{-57}$$

$$R(metastability\ \cap still\ metastable\ after\ time\ T_2) = 1 * 10^{-48}\ times\ per\ second$$

$$\therefore MTBF = 9.7 * 10^{47}\ seconds = 3 * 10^{40}\ years$$

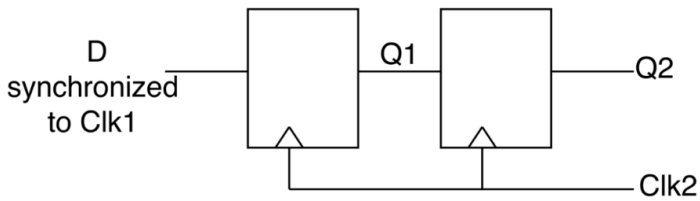
This tells us that if we waited for just 1 clock cycle after a metastability has occurred, the probability that we find the flip flop is still in metastability is $2 * 10^{-57}$. Or in other words, to find the flip flop is still in metastability after a complete clock cycle, we need to wait for $3 * 10^{40}$ years, which is astronomically long! This shows that the solution for metastability is to wait for one clock cycle after metastability and then sample the data safely.

How to handle/solve metastability?

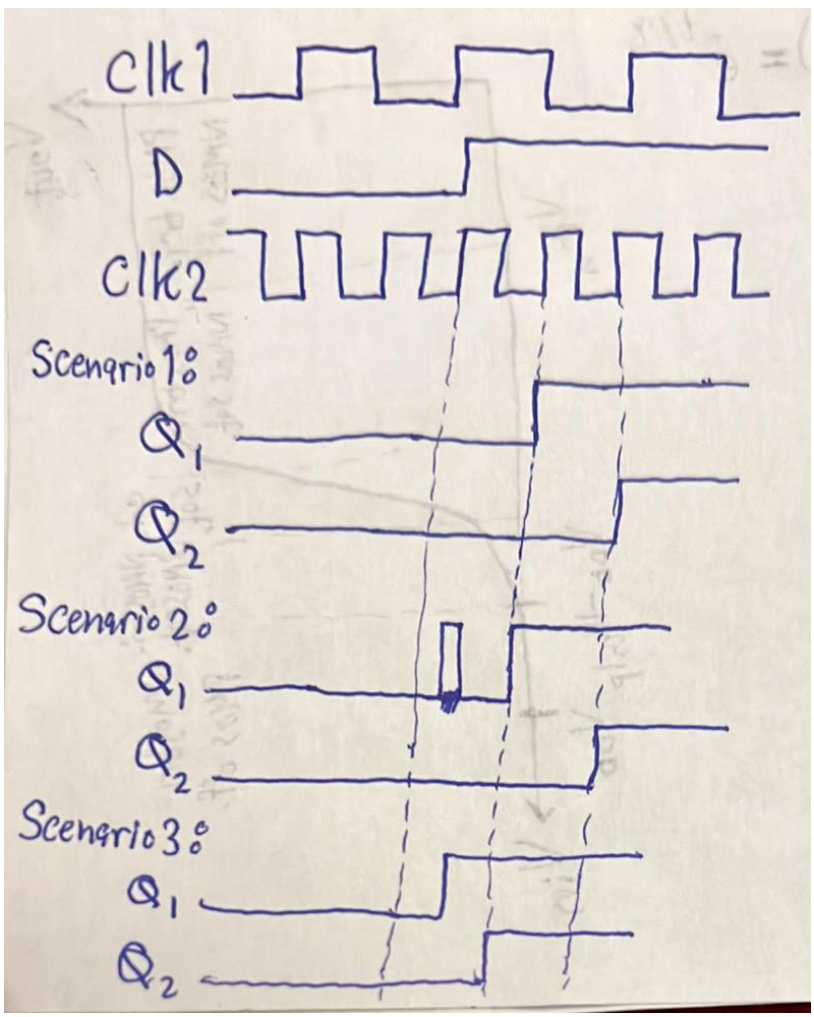
So, we now know that whenever metastability occurs, we need to wait for one clock cycle and boom all our problems are solved. The question is: how can we wait that one clock cycle after metastability occurs? in other words, what hardware should I use so that whenever metastability occurs, we wait for one clock cycle before sampling the received data?

Bit Synchronizer (Double Flop Synchronizer)

If a single bit is crossing from domain 1 to domain 2, then we use a double flop synchronizer as shown below:



We want to check that this double flop synchronizer would actually solve the problem of metastability. Accordingly, we will assume that D has changed in clock domain 1 in a way that caused Q_1 to become metastable, and then we observe how Q_1 and Q_2 would behave.

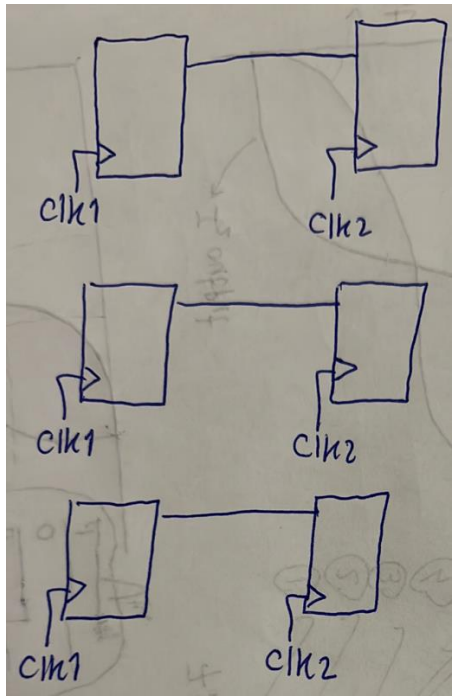


Notice that in all scenarios, Q_2 transition in a synchronous way. That is, it does not transition in a metastable way. Is it possible that Q_2 becomes metastable? the only way Q_2 can become metastable is that Q_1 remains metastable until the next clock edge. We have just said that the probability of such a thing is extremely small. Thus, it is almost guaranteed that Q_2 will never become metastable and hence it is ready to be passed to clock domain 2! See? this double flop synchronizer has indeed solved the problem!

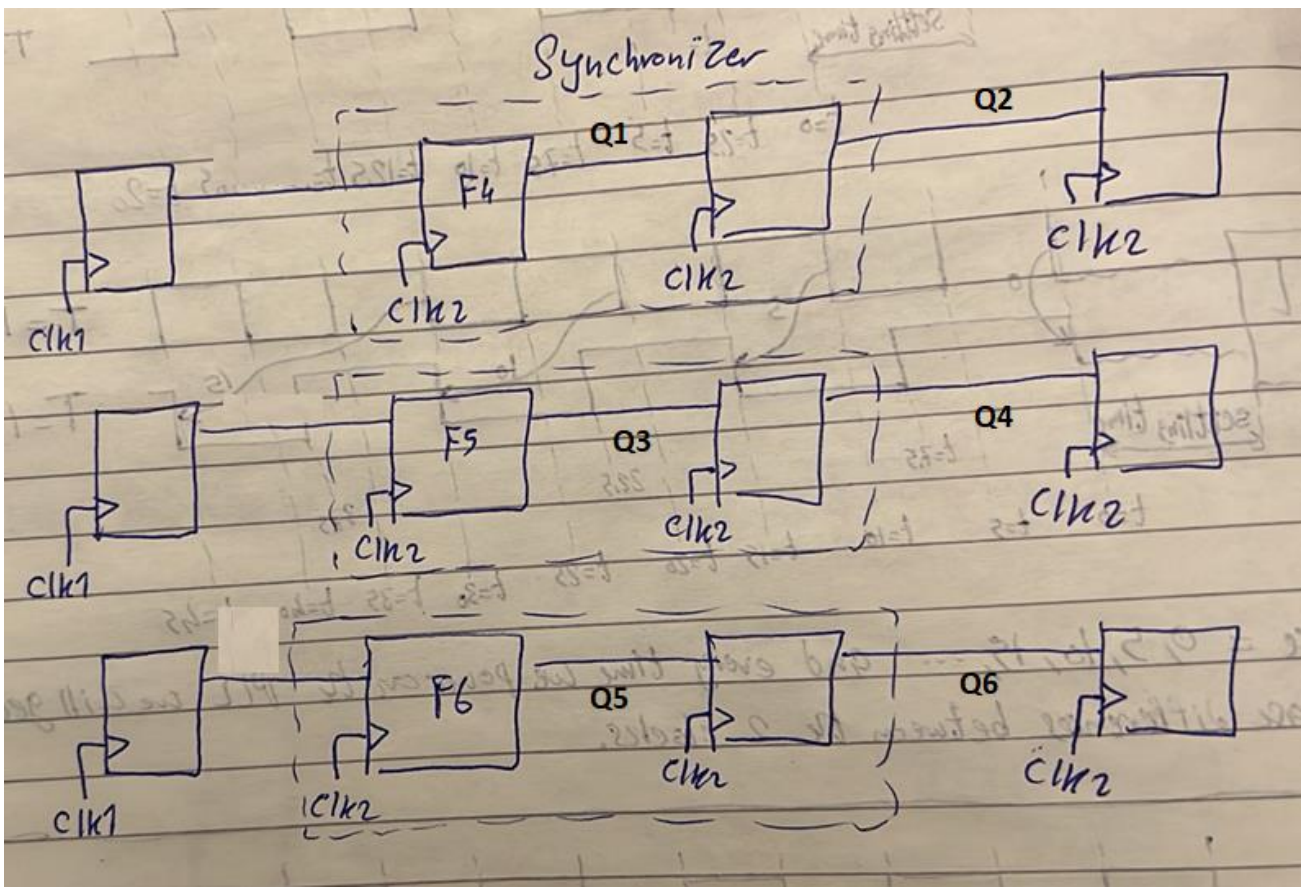
However, notice that there is a one clock cycle of uncertainty. That is, Q_2 might rise at a certain edge or the next edge, depending on where exactly the violation occurred on Q_1.

Bus Synchronization

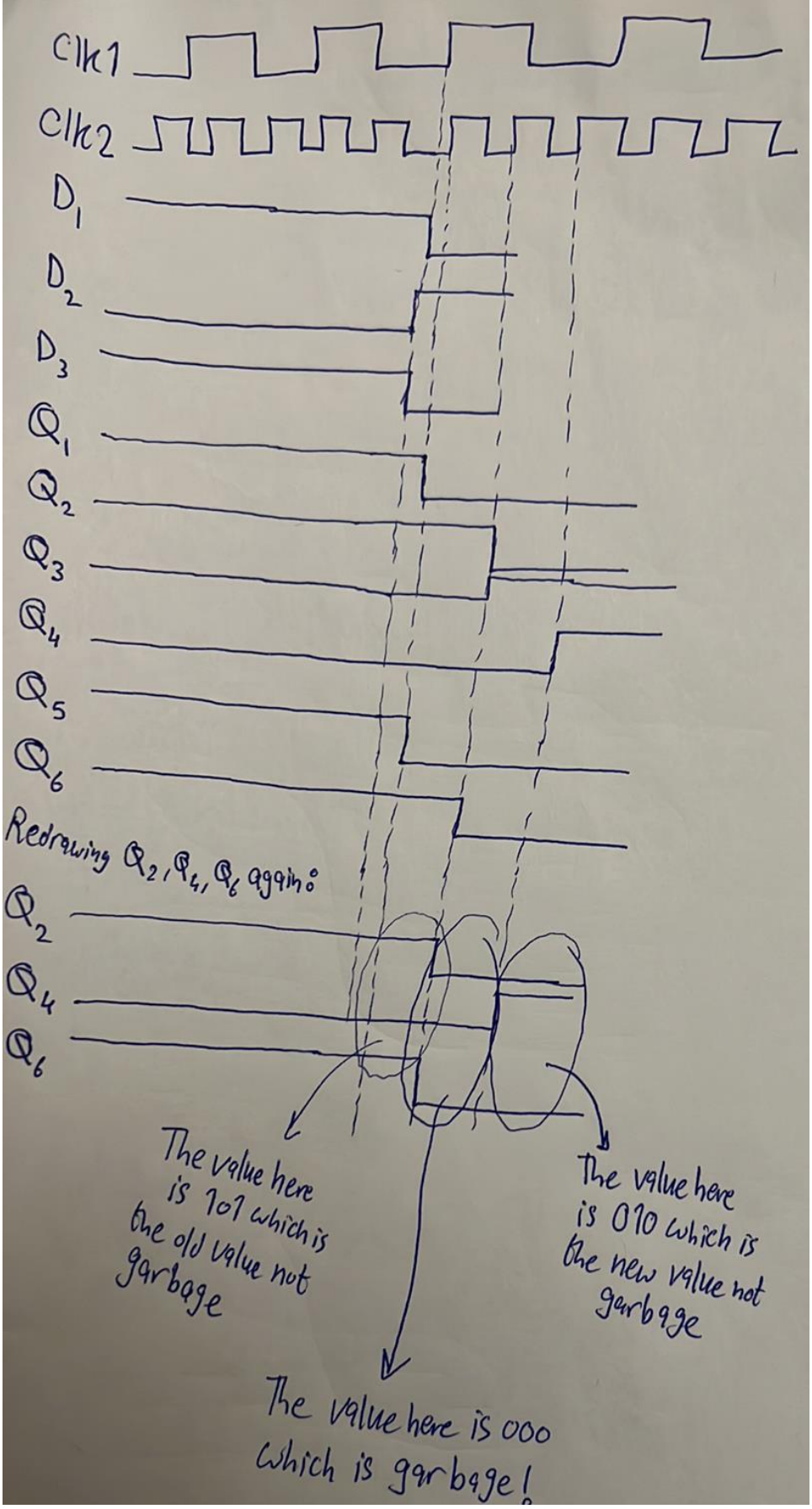
Now we want to transfer a bus from domain 1 to domain 2 as shown below. Bus means that a group of bits has a meaning when you sample them all together, one bin on its own does not have a meaning.



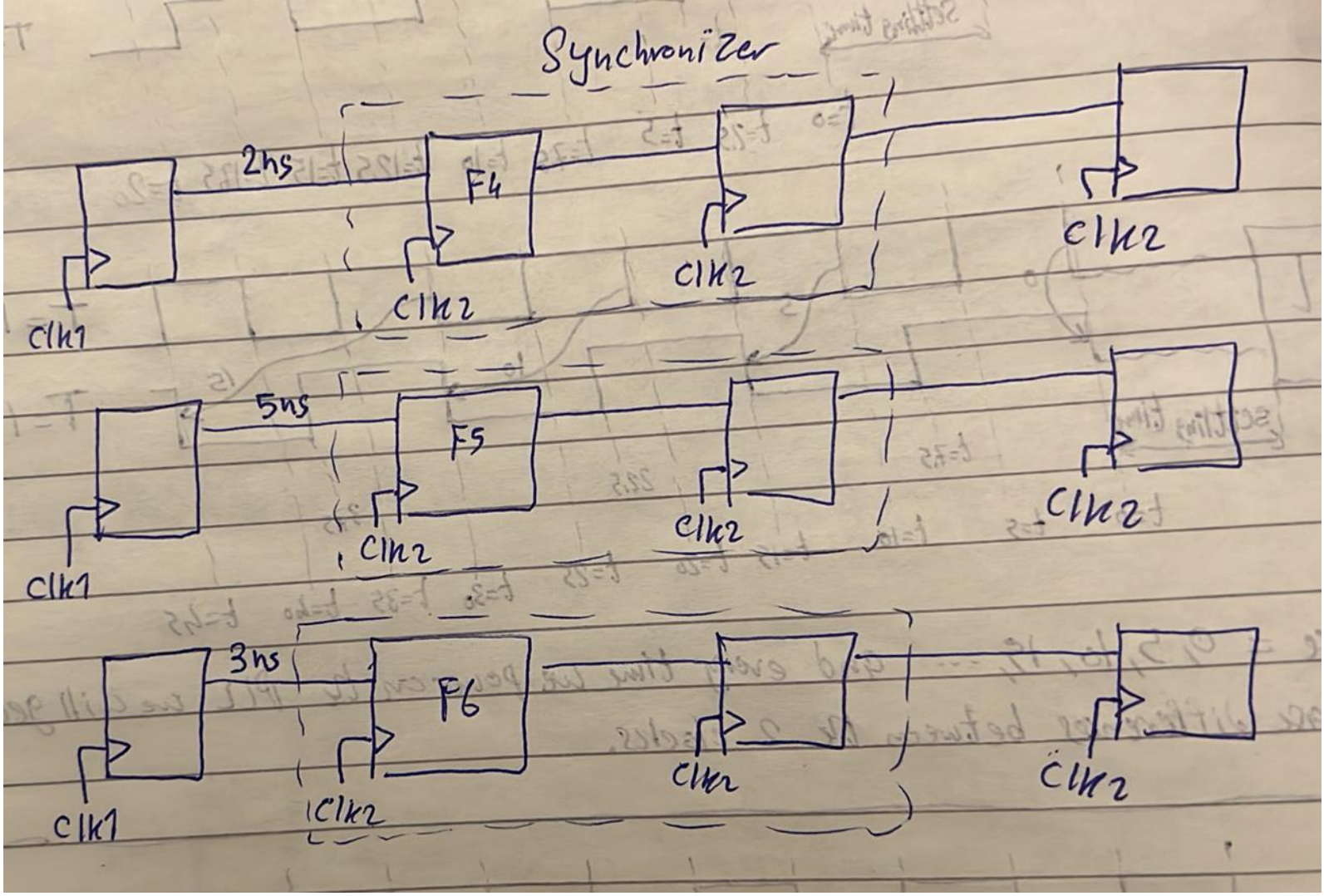
A trivial solution to do so is to use multiple double flop synchronizers as shown below.



However, this would lead to a complete failure of the bus. Why? because these double flop synchronizers are not guaranteed to behave similarly. That is, $F5$ for example might become metastable, while $F4$ and $F6$ do not become metastable. In that case, the first and the third bit will arrive safely at the receiver, while the second bit will have an uncertainty of one clock cycle. That is, it might reach at the same time as the other two bits or delayed by one clock cycle. In case it was delayed, this will result in the bus having a garbage value at the receiver. For example, assume the bus was carrying the value 101. Then, domain 1 changed it to 010. Assume the first and third bit arrived safely without metastability at the receiver while the second bit suffered from metastability. Here is what will happen:



Thus, using multiple double flop synchronizers to transfer a bus from one domain to another is not allowed because this allows garbage data to be present on the bus from time to time. But why might the synchronizers' flip flops behave differently? because the wiring delay can be different. Following on the previous example, assume the wiring delays are as shown below. This shows that the data arrives at $F5$ later compared to $F4$ and $F6$. Accordingly, setup time violation can occur on $F5$ will not on $F4$ and $F6$, as was the case in the previous example.

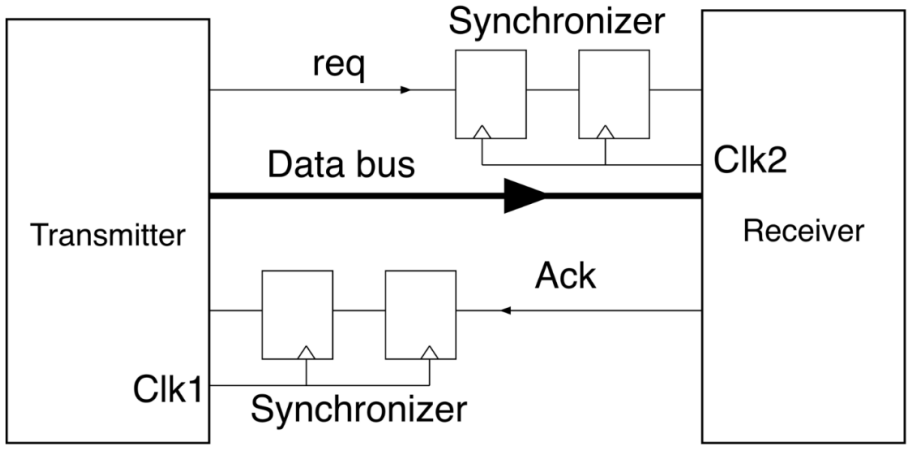


Watch this from 1:45 to 3:09 for another example.

Using multiple double flops to synchronize a bus can only be applicable if the bus was a gray counter. Why? because we know that each gray count differs from the previous count in just 1 bit. Accordingly, when this 1 bit is transferred from domain 1 to domain 2, it might or might not suffer from metastability. If it doesn't suffer from metastability, then the old count will remain at domain 2 and then the new count will arrive after 2 clock edges. If it suffered from the metastability, then the old count will remain at domain 2 and then the new count will arrive after 3 clock edges. See? in both cases, no garbage data was found on the bus as was the case before.

Handshaking Synchronizer

A bus can generally be transferred from one domain to another using a handshaking protocol as shown below:

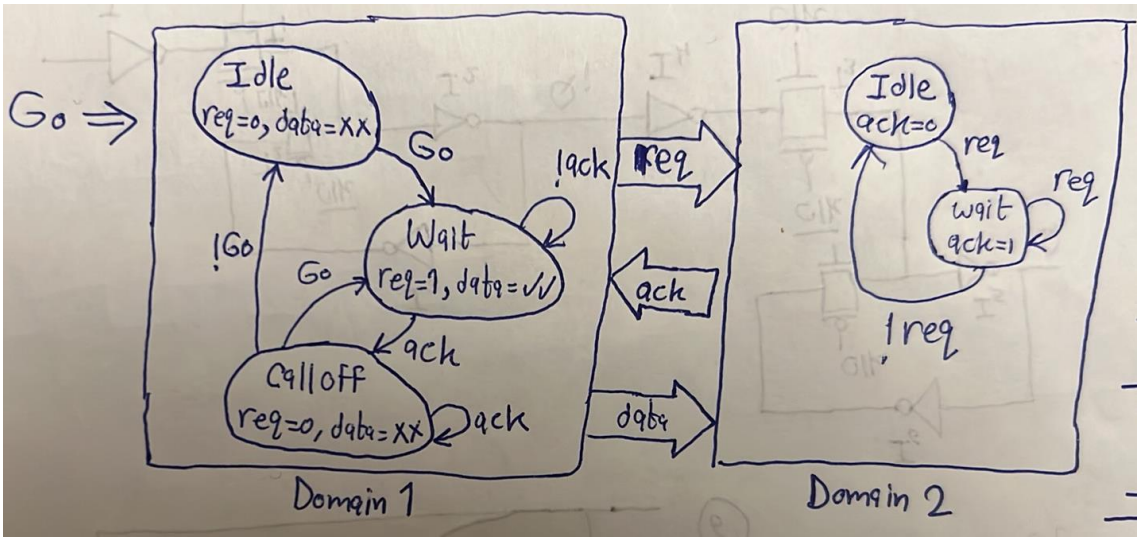


The flow is as follows:

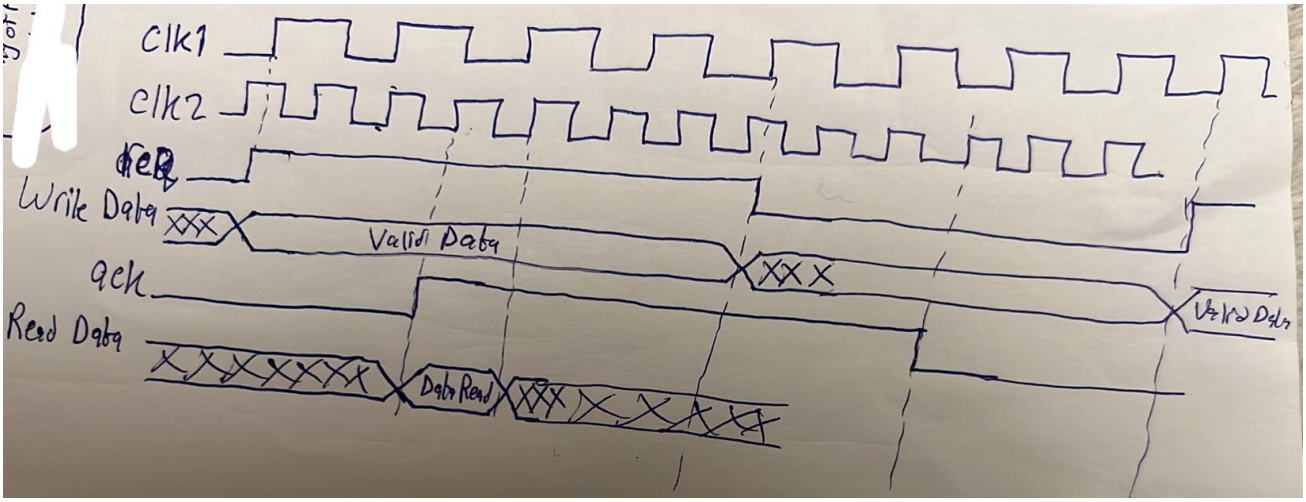
1. A data word is prepared on the bus synchronous with clk1. Simultaneously, req is raised on clk1.
2. Req is synchronized to the receiver clock clk2 using a double flop synchronizer. The receiver will either read the req on the second or third edge of the clock.
3. Because req and data were generated on the same clock, when req is synchronized at the receiver, it is safe to read the entire data bus. Thus, the receiver registers the data bus.
4. The receiver raises the “ack” flag on clk2, indicating the word has been read.
5. The ack flag is synchronized to the transmitter clock using a double flop synchronizer. The transmitter reads the ack flag on the second or third edge.
6. When the transmitter reads ack, it lowers the req signal, indicating it now understands the receiver has registered the data and it is now ready to receive a new word.
7. The falling req signal is synchronized to the receiver. The receiver then lowers the ack signal, indicating it understands the transmitter might send a new word.
8. The falling ack signal is synchronized to the transmitter, indicating the receiver is ready to receive a new word.
9. The cycle can now restart, and a new word can be transmitted.

Notice that this means that the data bus is not entering directly to the input of flip flops at the receiver. There is a MUX before each flip flop that chooses whether to pass the data bus or just maintain the old value. The receiver should control these MUXs such that it passes the data from data bus when it is safe to read.

This can be described by state diagrams as shown below:



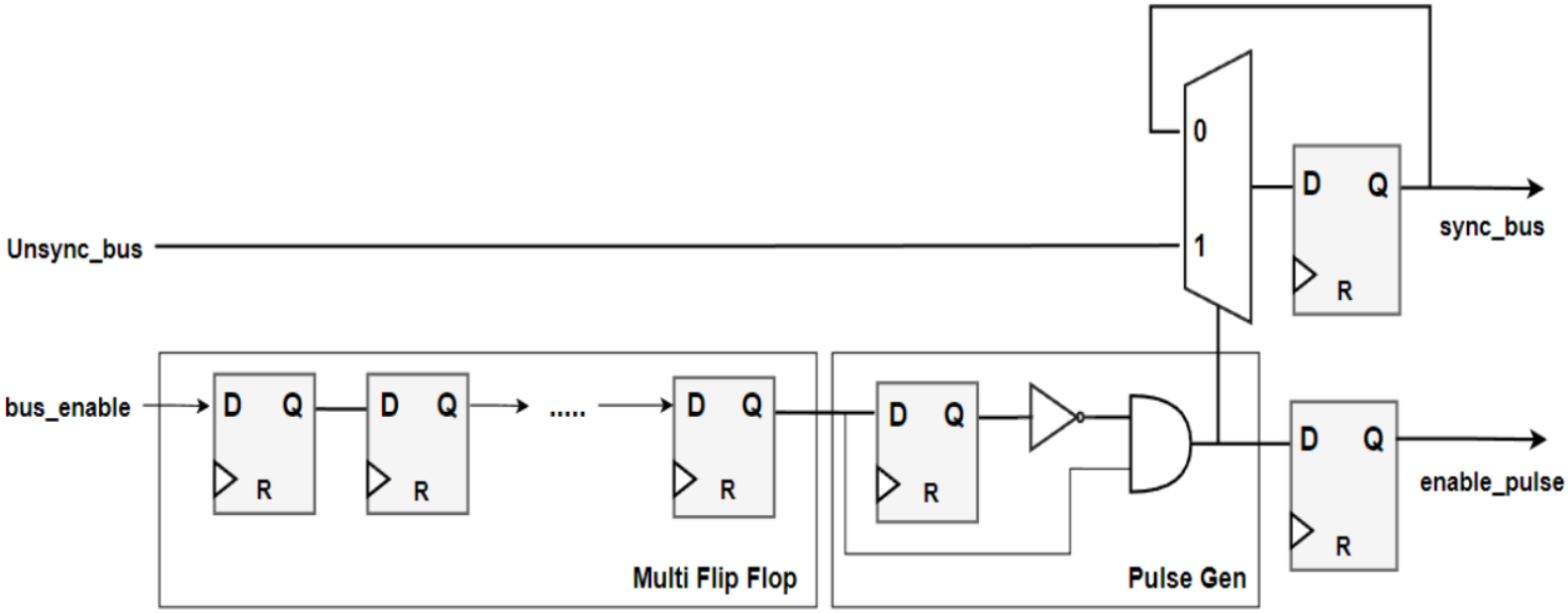
An example of a timing diagram is also shown below:



Notice that too many clock cycles are wasted just to transmit one word of data from domain 1 to domain 2. As a result, using handshaking synchronization is only applicable when the transmitter and receiver communicate with each other occasionally.

MUX-Based Synchronizer

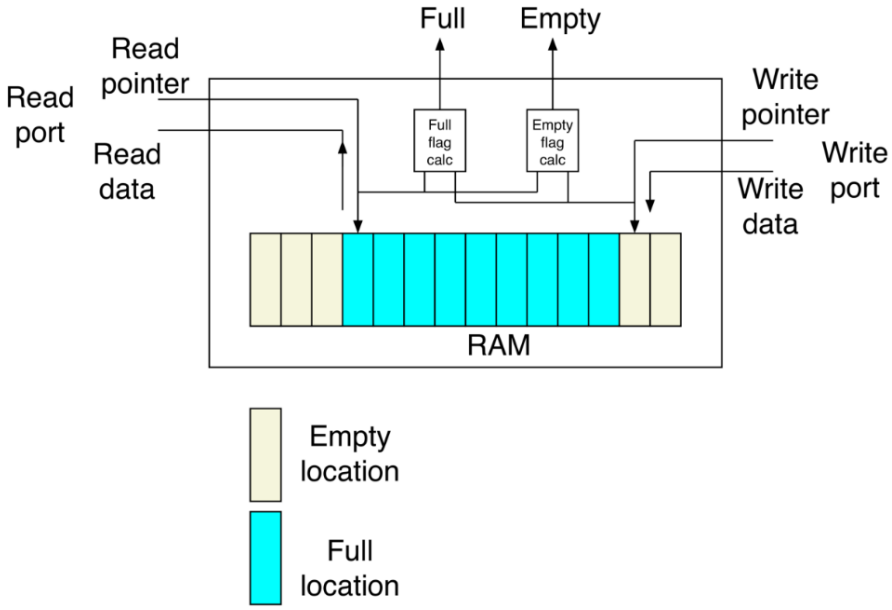
Another method that has less latency is using the mux-based synchronizer shown below. Its main idea depends on sending a synchronized enable signal plus an unsynchronized bus from domain 1 to domain 2. Ka2enaha heya heya el method el fatet bs el far2 en ka2en fih request signal w mafeesh ack signal. Fa hena el denya hatemshy ezay? el transmitter hay7ot el data bta3to 3la el bus w yerfa3 el enable be 1. El enable signal de hat get synchronized 3la el destination domain such that en el "Pulse Gen" da hayedmanly en el enable signal de hatewsal 3and el receiver 3la shakl pulse. 7ata lw enta ka transmitter fedelt sayeb el enable signal marfo3a be 1 fel domain beta3ak, fa heya hatewsaly ka receiver 3obara 3an pulse le modet 1 clock cycle fel domain beta3y. Athna2 el 1 cycle de, el data hat3ady men el mux and will be registered safely at the receiver. El shart hena 3la el transmitter howa eno mayghayarsh el data el 3al data bus b3d ma yerfa3 el bus enable ela b3d shwaya 7elween. El shwaya el 7elween dool hayeb2o such that enohom yedmanoly en el MUX delwa2ty b2a bey select el sync_bus and hence a2dar aghayar ana el data el 3la el unsync_bus. Tyb el shwaya dool hayeb2o 2ad eh? 7asab el design bta3ak b2a. Also notice that the signal that controls the selection line of the multiplexer here is a signal coming from domain 1. This is unlike handshaking synchronization, where the control signal coming from domain 1 (req) just triggers the FSM of domain 2, and domain 2 controls the selection line of the MUXs.



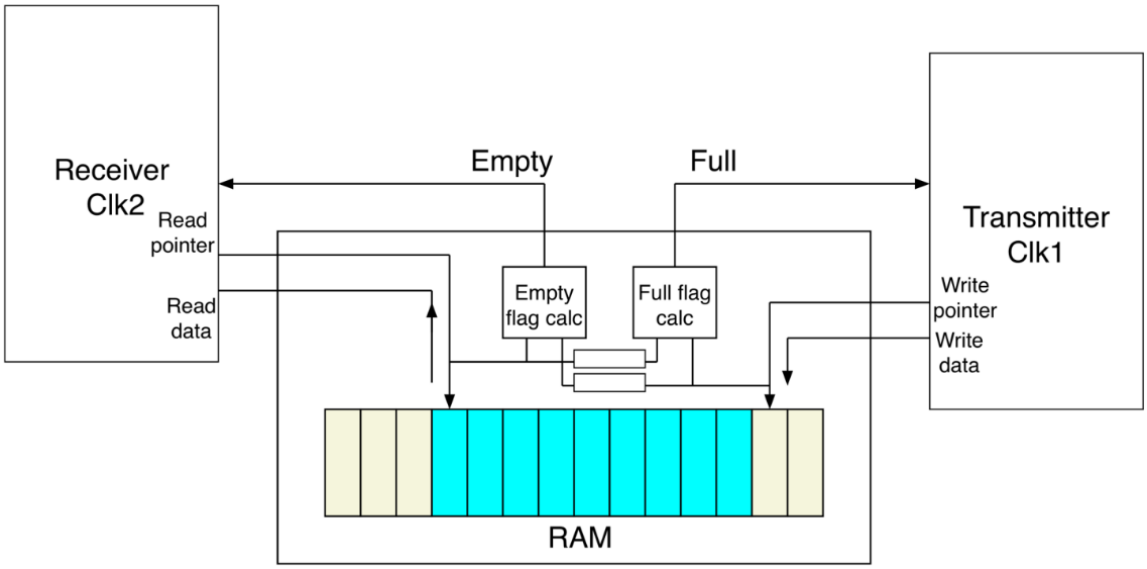
Fa notice en el method de fiha latency a2al compared to the handshaking synchronization, however it is less safe because the transmitter might change the unsync_bus while the multiplexer is enabled and hence timing violation might occur. Handshaking synchronization is the safest method to cross two domains. However, if a burst of data (i.e. many successive words) is to be transferred from domain 1 to domain 2, it is more efficient to use asynchronous FIFOs.

Asynchronous FIFO

- FIFOs can be generally used to cross from faster clock to slower clock. They may also be used when crossing from slower clock to faster clock with FIFO depth ≈ 1, but it is more common to use it in case of fast to slow crossing.
- The internal structure of a FIFO is shown below. Notice that it can be synchronous or asynchronous FIFO. Synchronous FIFOs are used when crossing between two synchronous (i.e. have constant phase relationship) clock domains. In this case, both read and write ports operate with the same clock. Asynchronous FIFOs are used when crossing between two asynchronous (i.e. have non-constant phase relationship) clock domains. In this case, each port operates with a separate clock.



- A dual-port RAM is used to store the data written in the FIFO.
- There is a write pointer and read pointer. The write pointer points to the location we will write in next. The read pointer points to the location we will read from next.
- Full flag is used to tell the write port (i.e. sender) whether the FIFO is full or not.
- Empty flag is used to tell the read port (i.e. receiver) whether the FIFO is empty or not.
- When both pointers point to the same location, this means that the FIFO is either full or empty. How to differentiate? this depends on the last update that caused equality. If you increment the write pointer and find that it becomes equal to the read pointer, then the FIFO is full. If you increment the read pointer and find that it becomes equal to the write pointer, then the FIFO is empty.
- Notice that to evaluate full and empty flags, we need to compare the read and write pointers. In asynchronous FIFOs, the full flag is synchronized to the write clock and the empty flag is synchronized to the read clock. Accordingly, before comparing read and write pointers for evaluating the full flag, we need to synchronize the read pointer to the write clock, and before comparing read and write pointers for evaluating the empty flag, we need to synchronize the write pointer to the read clock. This is shown below. This is shown below.



- Notice that the used synchronizers are multiple double flop synchronizers. Accordingly, both write and read pointers must be gray counters. However, we know that synchronizers cause latency with 1 or 2 clock cycles. Mesh momken el latency de te5aleny arfa3 el empty flag aw el full flag be tare2a ghalat? y3ny fel full flag masalan, delwa2ty el read pointer abl ma yegely bey3ady 3la synchronizer bey2a5aro 1 or 2 clock cycles. Mesh momken el ta25eer da ye5aly el transmitter fi wa2t men el aw2at yeb2a shayef en el FIFO lesa mesh full fi 7een en howa full w bel taly yekteb fi makan lesa mat2arash abl keda w teb2a moshkela? nafs el kalam 3and el empty flag, mesh momken el ta25eer fi en el write pointer yeroo7 lel read pointer ye5aly el

receiver fi wa2t men el aw2at yeb2a shayef en el FIFO lesa mesh empty fi 7een en howa empty w bel taly ye read data howa 2araha abl keda? watch this from 17:22 to 22:32.

- As we mentioned, asynchronous FIFOs are more efficient than handshaking synchronization in case of bursty data. Bursty ma3naha en el transmitter hayeb3at data keteer, then yeskot, then yeb3at keteer, then yeskot, and so on. El data el keteer el beyeb3at-ha fel mara el wa7da de esmaha burst.

Check Temsah's session from slide 18 for FIFO depth calculations and reset synchronizers.

Summary

- Two clocks are said to be in different domains when they do not have a constant phase relationship.
- Non-constant relationship occurs when the two clocks are from different clock sources or from the same source, but their frequencies do not have an integer relationship.
- Clock domain crossing causes metastability which may cause:
  1. longer t_{cq}.
  2. propagation of different values at the same time.
- The output of a metastable flip flop may transition after time longer than t_{cq}, do not transition and then transition one cycle later, or glitches then transition one cycle later.
- $MTBF|_{before\ synchronization} = \frac{T_1 T_2}{T_m}$, $MTBF|_{after\ synchronization} = \frac{T_1 T_2}{T_m} * e^{t/\tau}$
- single bit → double flop synchronizer.
- Gray counter → multiple double flop synchronizers.
- Two domains communicate occasionally → handshake synchronization or mux-based synchronization (depends on your choice between safety and latency).
- Bursty data and close frequencies → asynchronous FIFOs.